

Handoff — 500px-style Pulse Stats & Scoring

Date: 2026-06-06
Branch: feat/pulse-stats-500px (off admin)
For: dev ที่จะรับงานต่อ / apply migration / deploy

เอกสารนี้สรุป **ทุกขั้นตอน** ที่ทำในรอบนี้: ระบบคะแนน Pulse, การเก็บคะแนน, และ stats panel แบบ 500px พร้อม **สิ่งที่ dev ต้องทำ** ก่อนใช้งานจริง

1. TL;DR

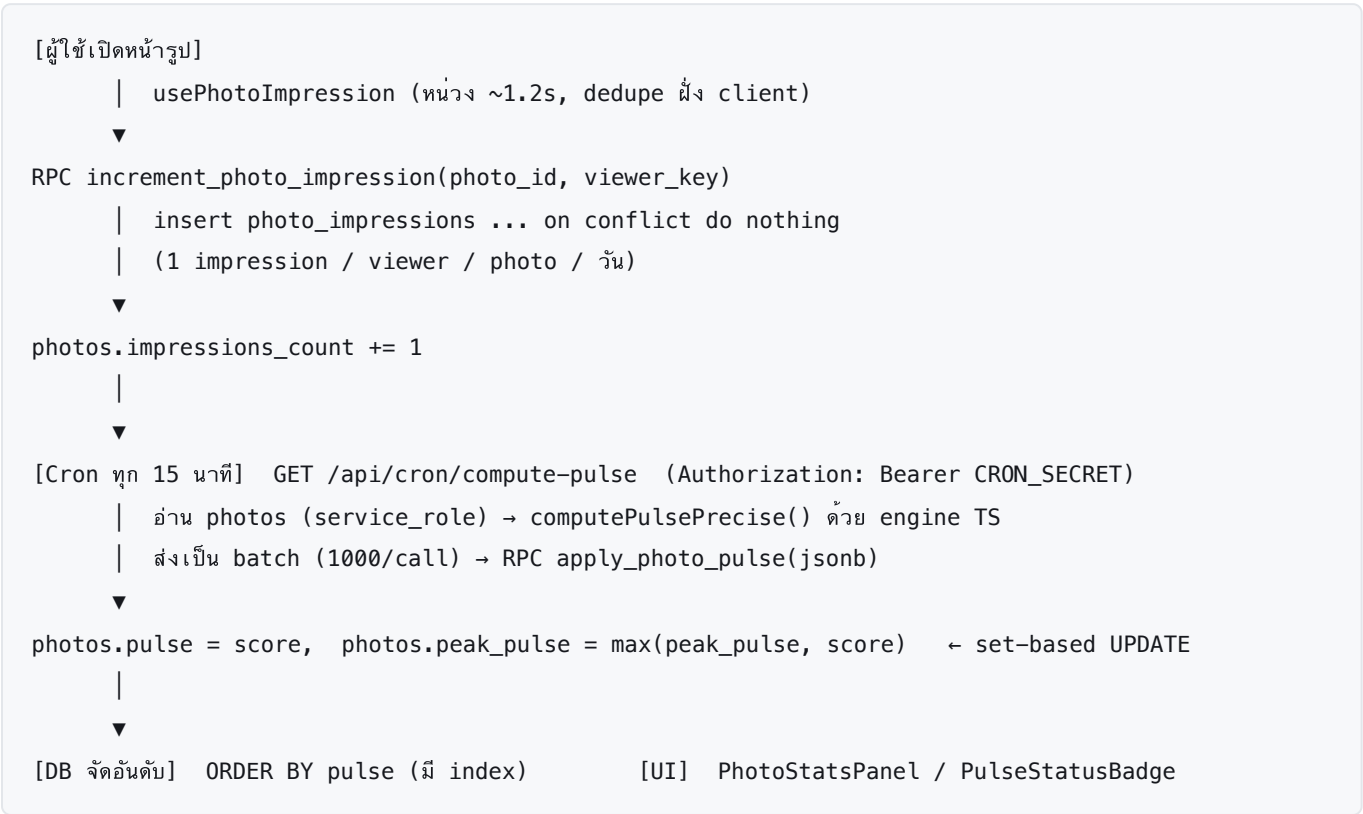
เป้าหมาย: ทำให้หน้ารูปโซเชียลมีเดียแบบ 500px — Likes / Impressions / Highest Pulse / Status (Popular) และทำให้ DB จัดอันดับด้วย pulse ได้

ปัญหาเดิมที่แก้:

- 1. คะแนน Pulse คำนวณใน JS ฟังก์ชัน client เท่านั้น → ไม่เคยถูกเก็บลง DB → ORDER BY pulse ไม่ได้
- 2. มี 2 สูตรไม่ตรงกัน: engine TS (pulse-engine.ts , ฉลาด) vs DB materialized view (0003 , naive) → คะแนนโชว์ ≠ คะแนนจัดอันดับ
- 3. impressions_count มีคอลัมน์แต่ ไม่มีอะไรไปนับ → exposure score = 0 เสมอ

วิธีแก้ (สถาปัตยกรรม): **engine TS = source of truth เดียว** → cron คำนวณแล้ว persist ลงคอลัมน์ → DB จัดอันดับจากคอลัมน์นั้น

2. Data flow



****Highest Pulse = peak_pulse **** — เพราะ pulse มี decay (ขึ้นแล้วตก) ค่าที่ภูมิใจคือจุดพีค จึงเก็บ max ตลอดอายุรูป

3. สิ่งที่เราสร้าง (ไฟล์ทั้งหมด)

Phase 1 — Impression tracking

ไฟล์	หน้าที่
supabase/migrations/0014_pulse_persistence_and_impressions.sql	ตาราง photo_impressions (dedupe by photo+viewer+day) + RPC increment_photo_impression (SECURITY DEFINER, grant anon/authenticated)
src/hooks/usePhotoImpression.ts	client hook — ยิง RPC หลังอยู่หน้า คสข ~1.2s, viewer_key เก็บใน localStorage, กันยิงซ้ำด้วย sessionStorage

Phase 2 — Persist pulse + peak

ไฟล์	หน้าที่
0014_...sql	เพิ่มคอลัมน์ photos.pulse + photos.peak_pulse (numeric 5,1) + index pulse desc / peak_pulse desc
supabase/migrations/0015_apply_photo_pulse.sql	RPC apply_photo_pulse(jsonb) — set-based UPDATE (batch), peak = greatest(...), grant service_role เท่านั้น
src/app/api/cron/compute-pulse/route.ts	cron route — auth ด้วย CRON_SECRET , ใช้ service_role client, คำนวณด้วย engine, เขียน batch 1000/call
vercel.json	ตั้ง Vercel Cron */15 * * * * → /api/cron/compute-pulse
src/lib/pulse-engine.ts	เพิ่ม computePulsePrecise() (ทศนิยม 1 ตำแหน่ง "99.4") โดย computePulse() เดิมไม่เปลี่ยน + pulseStatus() + PULSE_STATUS_THRESHOLDS

Phase 3 — UI

ไฟล์	หน้าที่
<code>src/components/photo/PhotoStatsPanel.tsx</code>	panel 1000px (Likes / Impressions / Highest Pulse / Status)
<code>src/components/photo/PulseStatusBadge.tsx</code>	badge สถานะ monochrome (Rising/Popular/Editors' Choice), undiscovered = ไม่ใช้
<code>src/app/photo/[id]/PhotoDetailClient.tsx</code>	ต่อ panel (mobile = main column <code>lg:hidden</code> , desktop = sidebar <code>hidden lg:block</code>) + เรียก hook + map <code>impressions/peakPulse/pickType</code>
<code>src/components/photo/PhotoCard.tsx</code>	ใส่ <code>PulseStatusBadge</code> ได้ชื่อข้างภาพ → ครอบคลุม grid
<code>src/lib/types.ts</code>	เพิ่ม optional field <code>impressions?</code> / <code>peakPulse?</code> / <code>pickType?</code> ใน <code>Photo</code>
<code>src/messages/{en,th}.json</code>	namespace <code>PhotoStats</code> (title/likes/impressions/highest_pulse)

4. ⚠ สิ่งที่ dev ต้องทำก่อนใช้งานจริง (REQUIRED)

4.1 Apply migrations

รัน SQL บน Supabase ตามลำดับ:

```
supabase/migrations/0014_pulse_persistence_and_impressions.sql
supabase/migrations/0015_apply_photo_pulse.sql
```

(ผ่าน Supabase SQL editor หรือ `supabase db push` ตาม workflow ของทีม)

4.2 ตั้ง env (ทั้ง local `.env.local` และ Vercel Project Settings)

```
SUPABASE_SERVICE_ROLE_KEY=<Supabase → Settings → API → service_role>
CRON_SECRET=<สุ่มสตริงอะไรก็ได้>
```

- `SUPABASE_SERVICE_ROLE_KEY` = key ลับฝั่ง server (อย่า prefix `NEXT_PUBLIC_`, อย่า commit)
- `CRON_SECRET` = Vercel จะแทนเป็น `Authorization: Bearer <CRON_SECRET>` ให้อัตโนมัติเมื่อ cron ยิง

4.3 Deploy → Vercel cron จะรันเองทุก 15 นาที

ทดสอบ local ด้วยมือ:

```
curl -H "Authorization: Bearer <CRON_SECRET>" \
  http://localhost:3000/api/cron/compute-pulse
# คาดหวัง: {"ok":true,"scored":N,"updated":N}
# ไม่มี header → 401 (ถูกต้อง)
```

4.4 (ควรรำ) เลิกใช้สูตร velocity เดิมใน 0003

supabase/migrations/0003_materialized_views.sql ยังคำนวณ pulse_score ด้วยสูตรเก่า (likes/hours) และ 0004 ใช้ค่านั้นจัดอันดับ/cashback
→ เปลี่ยนให้ photo_scores / ranking อ่านจาก photos.pulse (ที่ engine เขียน) แทน เพื่อให้เหลือ source เดียว

5. Status tiers (ปรับได้)

pulseStatus(pulse, pickType) ใน pulse-engine.ts :

```
pickType != none           → Editors' Choice
pulse >= POPULAR (35)      → Popular
pulse >= RISING  (32)      → Rising
else                       → Undiscovered (ไม้โชว์ badge)
```

🔴 **สำคัญ:** ตอนนี้ PULSE_STATUS_THRESHOLDS = { RISING: 32, POPULAR: 35 } เป็นค่า **demo-tuned** เพราะ seed data มี pulse แค่ 26–36
พอมีข้อมูลจริง (likes + impressions จริง ทำให้ช่วงคะแนนกว้างขึ้น) **ต้องปรับขึ้นเป็น ~55 / 80** ไม่งั้น "Popular" จะเฟื่องทางที่ผิด: เปลี่ยนเป็น **percentile-based** (top 10% = Popular) จะไม่ต้องจูนมือ

6. ⚠️ Known issue — Floor effect ที่ปลายล่าง (สำคัญต่อ ranking)

อาการ: รูปที่ 1 like กับ 4 likes โชว์ Highest Pulse = 19.0 เท่ากัน

สาเหตุ: engine clamp ขั้นต่ำที่ FL00R = 19 (ตั้งใจ — ไม่ให้มีคะแนนต่ำน่าเกลียด) → รูปที่คะแนนดิบ < 19 ถูกดันขึ้น 19 หมดประกอบกับ:

- **impressions = 0** (Phase 1 ยังไม่ deploy) → exposureScore = 0 → คะแนนยิ่งต่ำติดพื้น
- engagement เป็น log10 → ค่าต่ำถูกบีบใกล้กัน

ตัวเลขจริง (impressions=0, ไม่มี fav/comment, รูปสด):

likes	คะแนนดิบ	หลัง floor (ที่โชว์)
1	~7.5	19.0
4 (metadata คสย)	~22	22
4 (metadata ไม่คสย)	~17.5	19.0
10	~26–31	26–31
50	~43–48	43–48

รูปในรีพอร์ตได้ 19 เพราะ **impressions=0 + metadata ไม่คสย** (ไม่มี camera/lens/location) → 4 likes = ~17.5 < floor

ผลกระทบ: รูป engagement ต่ำ **กระจุกที่ 19 หมด** → จัดอันดับปลายล่างไม่ได้ (เสมอกัน เรียงไม่ออก) และ "Highest Pulse" หน้าตาเหมือนกัน

✅ **ทางแก้ที่แนะนำ (ทาง A) — แยก "ค่าโชว์" ออกจาก "ค่าจัดอันดับ"**

- **โชว์:** ใช้ค่าติด floor 19 เหมือนเดิม (พรีเมียม)
- **จัดอันดับ:** ใช้ค่าดิบ **ไม่ติด floor** + tiebreak

ทำจริง:

1. เพิ่มฟังก์ชัน engine แบบไม่ติด floor (เช่น `computePulseRaw()` — เหมือน `computePulsePrecise` แต่ไม่ clamp ขึ้นต่ำ)
2. เพิ่มคอลัมน์ `photos.pulse_raw numeric` — cron เขียนทั้ง `pulse` (floor, โชว์) และ `pulse_raw` (rank)
3. ranking/leaderboard ใช้ `ORDER BY pulse_raw DESC, likes_count DESC, uploaded_at DESC`

ทางเลือกอื่น

ทาง	ทำอะไร
B	เปิด impressions tracking (Phase 1) → exposure ทำงาน คะแนนกระจายเอง
C	ลด FL00R (เช่น 5) — ค่าจริงโชว์ แต่เสีย "ขั้นต่ำสวย"
D	บังคับ metadata (camera/lens/location) ตอนอัปโหลด → เพิ่มความต่าง + คุณภาพข้อมูล

แนะนำ **A + B + D** ร่วมกัน: A แก้ ranking กันที, B ทำให้คะแนนกระจายตามจริง, D ยกคุณภาพ input

7. Open items / TODO

- ❑ **Apply migration 0014 + 0015** (ดู §4.1)
- ❑ **ตั้ง env** `SUPABASE_SERVICE_ROLE_KEY` + `CRON_SECRET` ใน Vercel (§4.2)
- ❑ **Retune status thresholds** หรือเปลี่ยนเป็น percentile (§5)
- ❑ **Floor effect** — ทำ "ทาง A" (เพิ่ม `pulse_raw` ไม่ติด floor สำหรับ ranking + tiebreak) (§6)
- ❑ เลิกใช้ velocity formula ใน 0003 , ให้ ranking อ่านจาก `photos.pulse` / `pulse_raw` (§4.4)
- ❑ (option) status badge ใน **mobile masonry tile** (`MobileExplore` / `MobileHome` feed) — ตอนนี้ใส่แค่ `PhotoCard` (desktop grid)
- ❑ **Pre-existing type errors** ใน `src/app/auth/callback/route.ts` (`nextUrl` does not exist on `Request`) — ติดมาก่อนหน้า ไม่เกี่ยวงานนี้ แต่ทำ `next build` fail ได้ ควรแก้ (ใช้ `NextRequest` แทน `Request`)
- ❑ **Voyageur → Traveller leftovers:** ยังมีคำตัวพิมพ์ใหญ่ `VOYAGEUR` ตกค้าง (`messages exclusive` , `SideMenu.voyageurs` , `explore subtitle/aria-label`) — ดู branch `feat/rename-traveller`

8. สถานะ branch (รอบงานนี้ทั้งหมด)

Branch	เนื้อหา	สถานะ
feat/pulse-stats-500px	งานในเอกสารนี้ (Pulse stats Phase 1-3 + batch/mobile/badge)	push แล้ว, ยังไม่ merge
feat/footer-unify	footer ใหม minimal "RANKING - SEASON 01" ทุกหน้า	merge เข้า admin แล้ว
feat/about-rewrite	เขียนหน้า About ใหม่ (Gography Photo Ranking)	merge เข้า admin แล้ว
feat/rename-traveller	rename Voyageur → Traveller ทั้ง UI + "การจัดอันดับ"	merge เข้า admin แล้ว (เหลือ leftover §6)

production = branch main (Vercel) · admin = integration branch

9. อ้างอิง

- Spec อัลกอริทึม: docs/specs/pulse-algorithm.md
- Engine: src/lib/pulse-engine.ts (พารามิเตอร์ทั้งหมดอยู่ใน PULSE_PARAMS)
- สูตร engine (ย่อ): pulse = clamp((engagement_log + exposure + metadata + pick) × decay, 19, 100)
- DB schema: supabase/migrations/0001_init_schema.sql (photos, votes), 0003 (views — กำลังจะ retire)

Appendix A — Pulse Scoring v2: Ecosystem-relative model (ข้อเสนอ, ยังไม่ build)

เป็น direction ที่ควรเลือกก่อนสร้างต่อ — แก่ข้อจำกัดเชิงโครงสร้างของ v1 (absolute) ที่เจอจาก §6

ปัญหาของ v1 (absolute scoring)

คะแนนเป็นเป๋าทายตัว (≈10,000 likes ถึงได้ 100) ไม่สนขนาดแพลตฟอร์ม → ผลที่ตามมา:

- เว็บคนน้อย → โหวตไม่ถึง → รูปถูกระงับ 19-40 (ไม่มีใคร "ชนะ" แบบน่าภูมิใจ)
- เว็บคนเยอะ → ป้าย Popular เพ้อ (threshold ตายตัว)
- รูป viral → ดันที่ 100 แยกที่ 1 ไม่ออก

หลักการ v2

วัด "เก่งแค่ไหนเทียบกับสนามตอนนี้" ไม่ใช่ "ได้ทั่วโลก" → คะแนน/อันดับ สัมพันธ์กับ ecosystem ณ ขณะนั้น

ตัวแปร ecosystem (ทุกตัวเชื่อมกัน)

ตัวแปร	บทบาทในคะแนน
การกดไลก์ (votes)	สัญญาณหลัก (ตัวตั้ง)
คนดู (views/impressions)	ตัวหาร → ใช้ *อัตรา* ไลก์/วิว ไม่ใช่ยอดดิบ
จำนวนคนลงรูป (supply)	ขนาด "สนามแข่ง" → ใช้ทำ percentile
active users (demand)	liquidity → ใช้ทำ confidence + normalize ต่อช่วงเวลา

โมเดล 3 ชั้น

1. **Raw** — likes, favorites, comments, views ต่อรูป
2. **Normalize เทียบ field ปัจจุบัน** — เทียบกับค่าเฉลี่ยของรูปในช่วงเดียวกัน → ปรับตามขนาดเว็บอัตโนมัติ
3. **Confidence (Bayesian)** — รูป vote น้อย ดึงเข้าหาค่ากลางจนกว่าจะมีข้อมูลพอ (กันฟลัค 1 ไลก์/1 วิว)

สูตรแนวคิด

```
field_mean = ค่าเฉลี่ย engagement ของรูปในช่วงเดียวกัน (season หรือ rolling 7d)
relative    = engagement_photo / field_mean                # ทำได้ก็เท่าของค่ากลาง
adjusted    = bayesian_shrink(relative, n_votes, prior)    # vote น้อย → ดึงเข้าหา 1.0
rank        = percentile(adjusted) ของทั้ง field + tiebreak(likes → uploaded_at)
status      = ตาม percentile (top 5% = Popular, top 20% = Rising)
display     = map(percentile) → 19-100 (เว็บเล็กผู้นำก็โชว์สูง ดู "มีชีวิต")
```

3 สถานการณ์ (absolute vs v2)

สถานการณ์	v1 absolute	v2 ecosystem-relative
เว็บเพิ่งเปิด (ดีสุด 8 ไลก์)	ทุกคน 19-24	ผู้นำสนามโชว์ ~90 เว็บดูคึก
เว็บโต 120 รูป	Popular 43 รูป (เฟี้ยว)	top 5% = 6 รูป ไม่เฟี้ยว
รูป viral ตัน	2 รูป = 100 แยกไม่ออก	tiebreak → #1 ชัด
รูปใหม่ฟลัค 2 ไลก์/2 วิว	อาจพุ่ง	Bayesian กดไว้จนพิสูจน์

Decision points (ต้องเลือกก่อน build)

- ☐ normalize window: ต่อ **season (4 เดือน)** หรือ **rolling 7 วัน**?
- ☐ confidence: prior ก็โหวต/วิว ถึงเชื่อเต็ม (เช่น Bayesian C≈20)
- ☐ display: โชว์ percentile-mapped (เว็บเล็กดูคึก) หรือ raw engagement?
- ☐ status cutoffs: top % เท่าไหร่ = Popular / Rising

สถานะ:

PROPOSAL — ยังไม่ build · ของที่ build แล้ว (branch feat/pulse-stats-500px) ยังเป็น v1 absolute + floor
แนะนำทำ v2 ก่อนเปิดให้คนใช้จริงจำนวนมาก (เพราะ absolute พังตอน scale ตาม §6 + 3 สถานการณ์)